



Designing Artificial Intelligence Products

Module 3: Artificial Intelligence Technology
Fundamentals: Deep Learning

September 16, 2026

Dr. Amanda Fetch

Learning Outcomes

- Identify examples of applications that employ deep learning algorithms.
- Identify the components of neural networks.
- Calculate the gradient descent of a neural network.
- Implement deep learning techniques in Python.
- Analyze simulated neural networks.
- Calculate convolutions of neural networks.
- Categorize neural network algorithms.

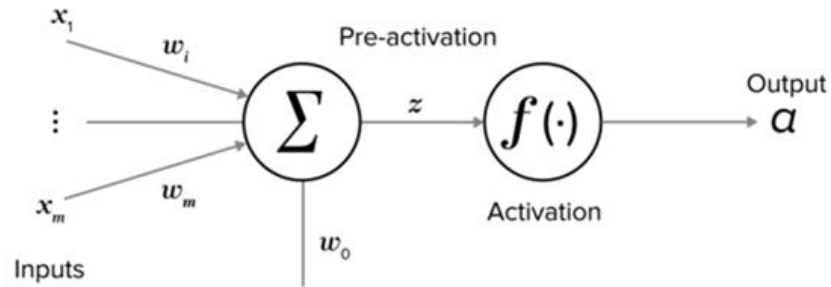


Machine Learning Methods

- **Neural Networks:** Neural networks are a type of machine learning model inspired by the way the human brain works. They consist of layers of connected nodes (like the circles in the image) that process and learn from data to make predictions or decisions.
- **Multi-layer Perceptrons (MLPs):** These are the most basic type of neural network. They have multiple layers of nodes: an input layer, one or more hidden layers, and an output layer.
- **Convolutional Neural Networks (CNNs):** These are a special type of neural network designed for processing grid-like data, like images.
- **Recurrent Neural Networks (RNNs):** These are a type of neural network that is particularly good at processing sequences of data.



Neural Networks



Potentially nonlinear activation function f produces output $a = f(z)$



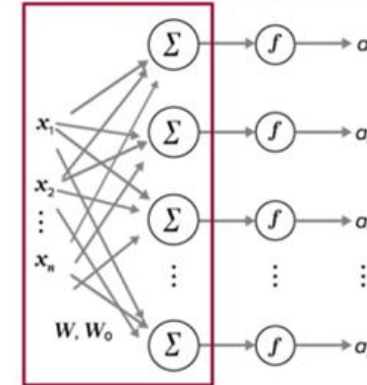
A neuron is a simple mathematical model inspired by biological neurons.

Inputs & Weights: Each input ($x_1, x_2, \dots, x_1, x_2, \dots$) is multiplied by a corresponding weight ($w_1, w_2, \dots, w_1, w_2, \dots$) before being summed.

Pre-Activation: The weighted inputs are summed to produce a pre-activation value (z).

Activation Function: This value is passed through a non-linear function to produce the output (a).

Multi-Layer Neural Network



Mathematical notation: Arrange the weight multiplications and summations in the matrix form $Z = WX + W_0$



Multiple neurons are organized into layers, with each layer processing the inputs. All inputs are connected to each neuron in a layer.

The operations within a layer can be expressed using a simple matrix equation above.

The network uses non-linear activation functions to map input vectors to output vectors, allowing it to learn complex patterns.

More layers and neurons increase the network's ability to model intricate relationships but also add to the number of parameters to be learned.

Model Coefficients: Example

Input, $M = 10$

Hidden nodes, $N = 5$

How many outputs do we have?

Output = 5

How many coefficients?

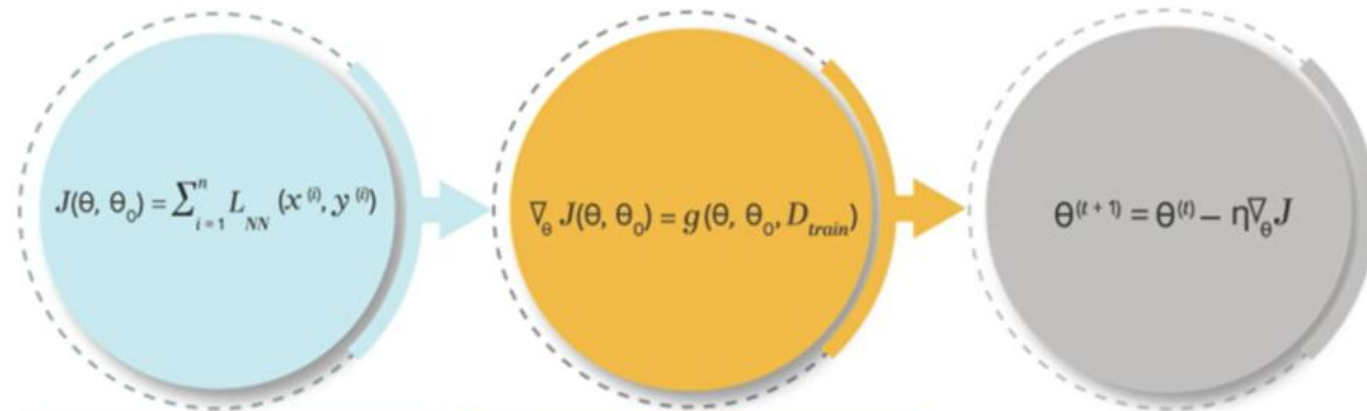
Number of coefficients = $M + 1$

$$\text{Model coefficient} = (10 + 1) \times 5 = 55$$

This number of coefficients together with nonlinearity gives a lot of expressive capacity if able to fit, train, or learn model coefficients.

MIT xPRO

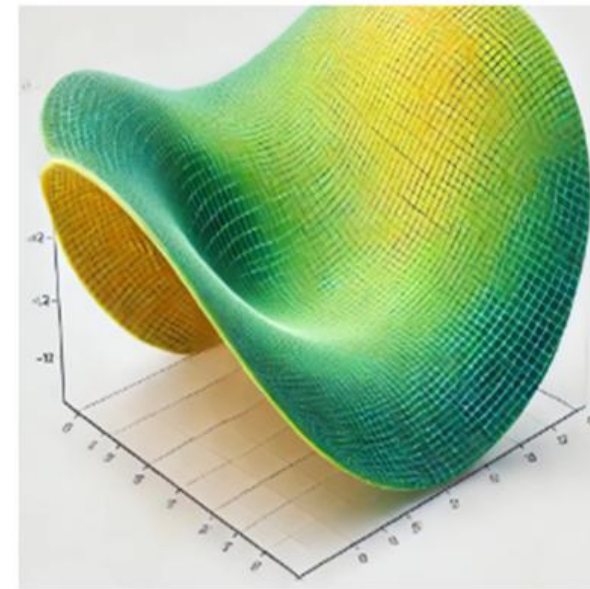
Gradient Descent



Express in terms of the loss $L(\cdot)$ associated with the prediction errors over n training points

Calculate the gradient with respect to the model coefficients

Modify the coefficients in a downhill direction of the gradient, with learning rate η

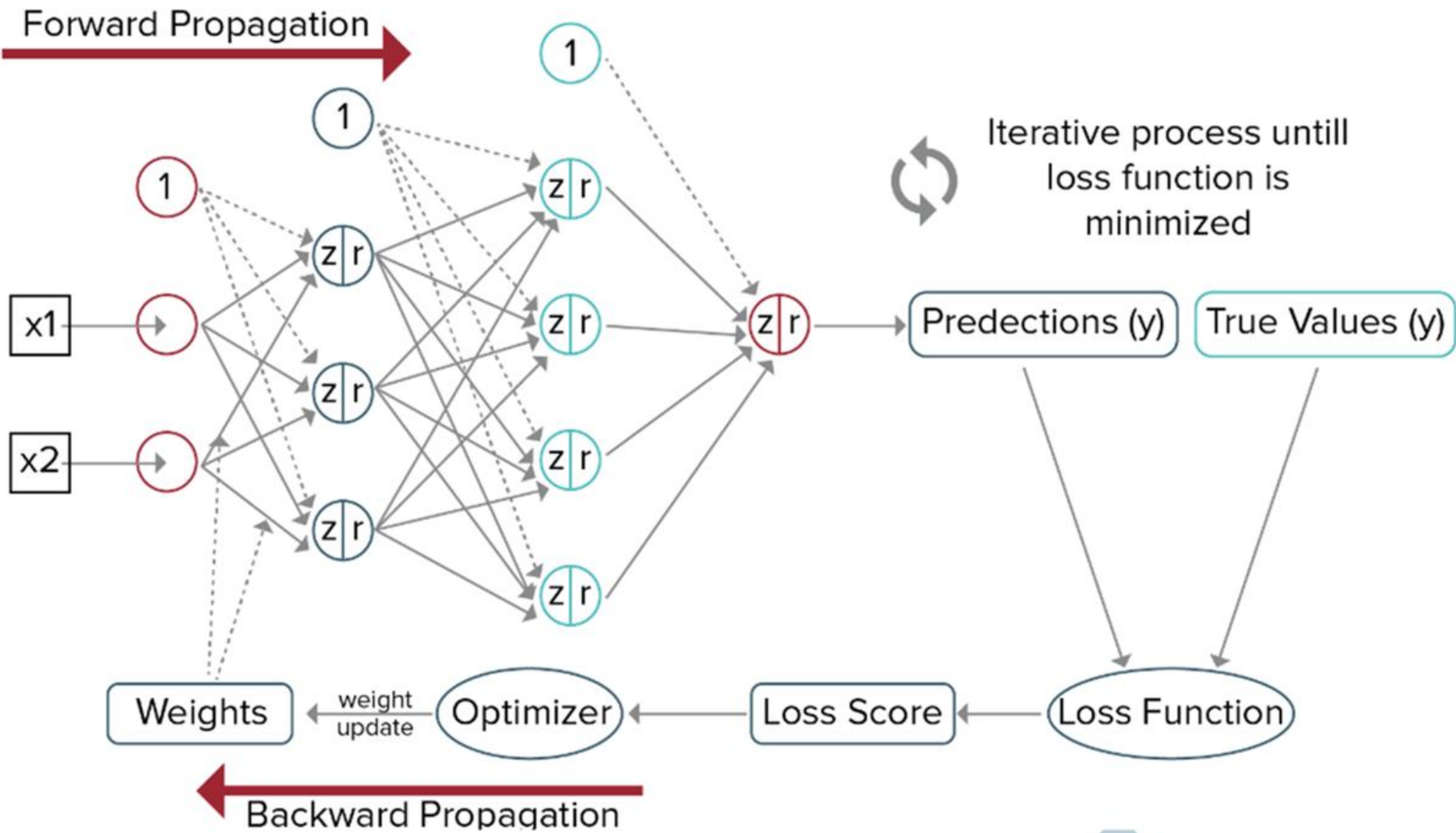


Stochastic gradient descent:

A randomized version of gradient descent

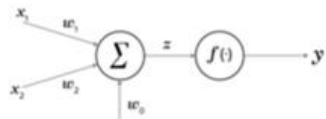
Batches of one or more data points are sampled from the training set to calculate the gradient and update the model

Number of batches for each epoch of training are considered until a criteria is met

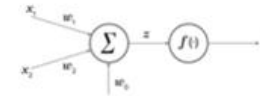




Gradient Descent Example



- Linear activation function, $f(z) = z$
- Initial weight, $w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$

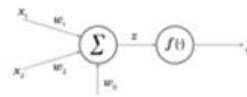


What is the predicted output, $\hat{y}$?

What are the gradients?

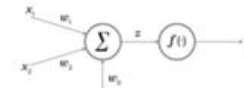
What are the new weights?

What is the new prediction with the new weights?



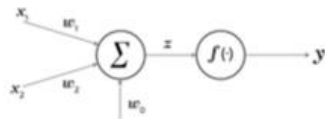
What is the predicted output, $\hat{y}$?

- $\hat{y} = 0$ – weights are 0
- $\hat{y} = 0 + 0 \times 1 + 0 \times 2 = 0$
- $\epsilon = \hat{y} - y = -4$

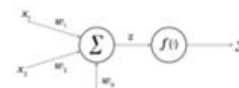


What are the gradients?

$$\begin{aligned} \nabla J w_0 &= 2\epsilon \frac{\partial \epsilon}{\partial w_0} = 2\epsilon = -8 \\ \nabla J w_1 &= 2\epsilon \frac{\partial \epsilon}{\partial w_1} = 2\epsilon x_1 = -8 \\ \nabla J w_2 &= 2\epsilon \frac{\partial \epsilon}{\partial w_2} = 2\epsilon x_2 = -16 \end{aligned}$$

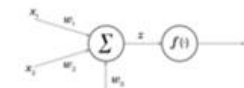


- Gradient descent with loss function $J = (\hat{y} - y)^2 = \epsilon^2$
- Learning rate, $\eta = 0.1$
- Training sample, $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$
- Training output, $y = 4$



What are the new weights?

$$\begin{aligned} w_{new} &= w - \eta \nabla J w = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} -8 \\ -8 \\ -16 \end{bmatrix} \\ w_{new} &= \begin{bmatrix} 0.8 \\ 0.8 \\ 1.6 \end{bmatrix} \end{aligned}$$

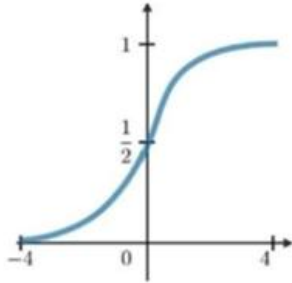


What is the new prediction with the new weights?

$$\begin{aligned} \hat{y} &= 0.8 + 0.8 \cdot 1 + 1.6 \cdot 2 = 4.8 \\ &\text{Closer to the training value 4} \end{aligned}$$

ANN Activation Functions

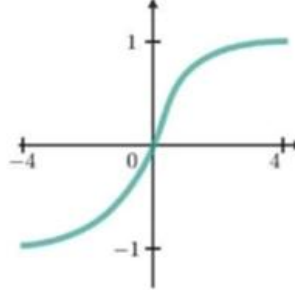
Sigmoid
$$(x) = \frac{1}{1+e^{-x}}$$



Ex. Email Spam Detection



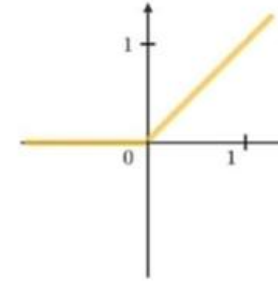
Tanh
$$(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



Ex. Stock Price Prediction



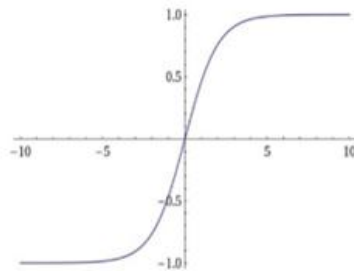
ReLU
$$(x) = \max(0, x)$$



Ex. Object in Image Recognition



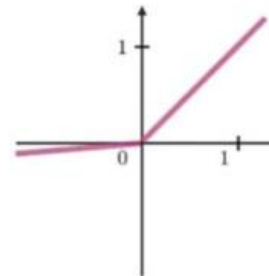
Softmax
$$(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$



Ex. Classify Handwritten Digits



Leaky ReLU
$$(x) = \max(0.01x, x)$$



Ex. Song Recommendations



Anatomy of an Artificial Neuron [Drag-and-Drop Activity 3.1]

Learning Outcome Addressed

2. Identify the components of neural networks.

This is a self-study activity and does not count toward the completion of the program.

Context

The fundamental unit or basic element in ANNs is the "neuron," based in part on a long history of analogies to the physiological neural brain elements and networks. This activity is designed for you to consolidate your knowledge about the different elements of a neuron.

Task

The image below depicts the structure of an artificial neuron. Match each component of the neuron with the corresponding name taken from the list below.

The average completion time for this activity is 10 minutes.

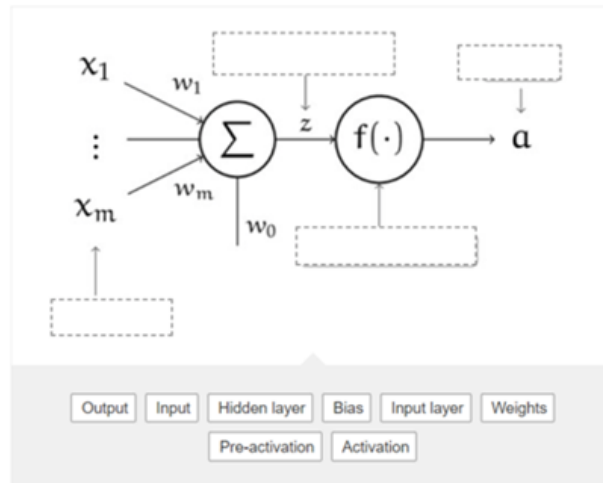
Student View

Edit question list

Reports



 You are viewing this activity in preview mode. Results will not be saved.

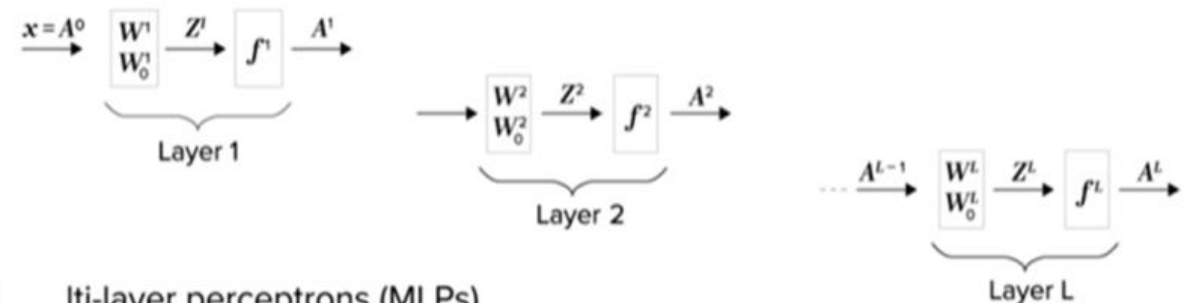
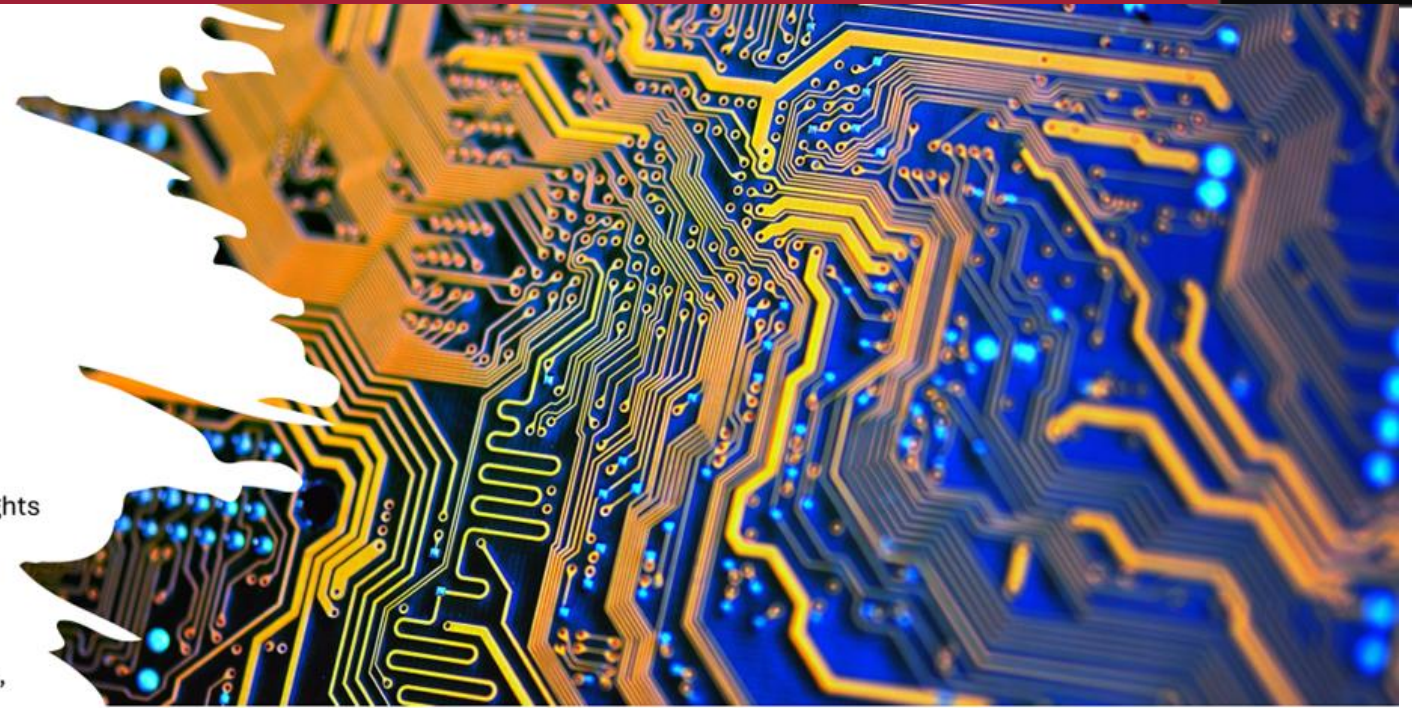


Loss Function Overview

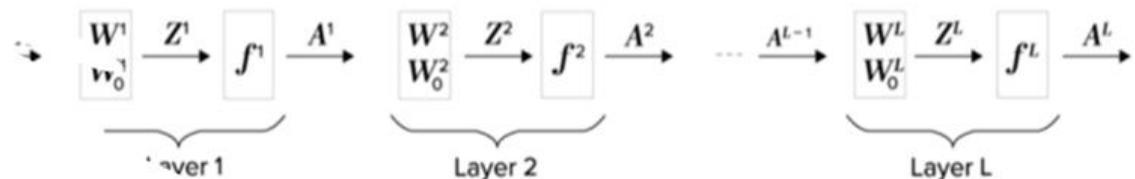
- **Customization:** Tailor the loss function based on the task.
 - **Regression:** Use squared loss for continuous predictions (e.g., temperature)
 - **Classification:** Use Negative Log Likelihood (NLL) for categorical outcomes (e.g., spam detection)
- **Key Components**
 - **Weights (w):** Parameters adjusted during training to minimize loss
 - **Data (d):** The dataset (training or test) used to evaluate the model
- **Example Applications**
 - **Regression:** Predict house prices; penalize large errors with squared loss
 - **Classification:** Identify cat vs. dog in photos; penalize misclassifications with NLL

Multi-Layer Perceptrons and Training

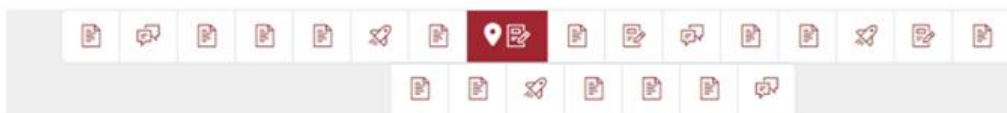
- **What are MLPs?**
 - Neural networks with multiple layers of neurons
 - Training involves complex calculations and tracking of weights across layers
- **Supervised Training**
 - Uses labeled data to minimize loss by adjusting weights
 - Gradient Descent iteratively adjusts weights to reduce loss, "learning" from data
- **Forward Pass**
 - Input data propagates through each layer, producing an output
- **Layer-by-Layer:**
 - In layer 1 Input is multiplied by weights, passed through an activation function.
 - Output of one layer becomes input for the next in subsequent layers.
 - In the final layer output is compared with the true label to calculate loss.
- **Example**
 - **Image Classification:** MLP classifies handwritten digits (0-9), transforming input through layers to predict the correct digit. Loss is computed based on prediction accuracy.



Multi-layer perceptrons (MLPs)



Run Single- and Multi-Layer Perceptrons in Python [Jupyter Notebook Activity 3.1]



Learning Outcome Addressed

4. Implement deep learning techniques in Python

This is a self-study activity and does not count toward the completion of the program.

Task

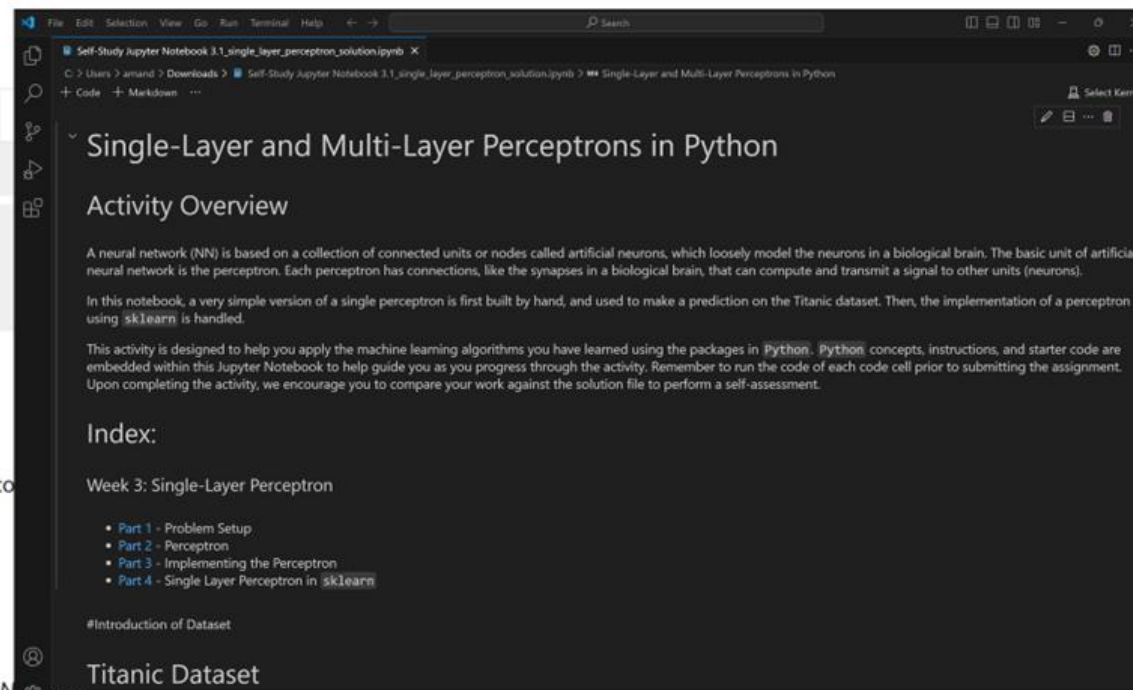
In this Jupyter Notebook activity, you will use the skills you've learned in the videos and mini-lesson to implement different types of perceptrons in Python.

Submission Instructions

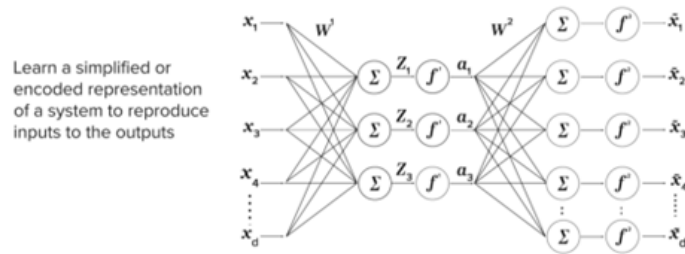
The average completion time for this activity is 30 minutes.

- After opening the activity in Codio, complete and run the code following the instructions in the Jupyter Notebook Activity.
- Important: There is nothing to submit in Codio or Canvas for this practice assignment.
- Once you've tried the activity, you can view the solution in [Google Colab](#). Find a guide to viewing your solution in Google Colab [here](#).
 - Download the solution here: [Self-Study Jupyter Notebook 3.1: Run Single- and Multi-Layer Perceptrons in Python](#)

This tool needs to be loaded in a new browser window



Autoencoders



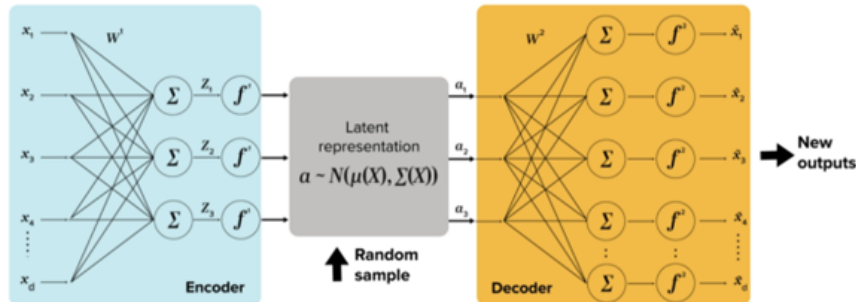
Autoencoders Simplified

Autoencoders: Neural networks that compress and reconstruct data, like simplifying a detailed picture into a basic sketch.

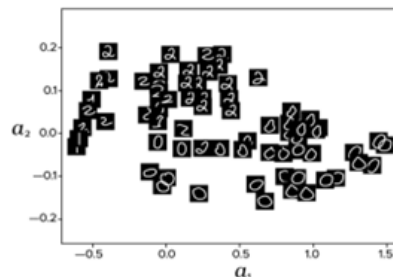
Encoder: Acts as a “shrinking machine,” reducing data to a smaller, essential “latent representation.”

Decoder: Rebuilds the original data from the compressed version, aiming to retain key details.

Variational Autoencoders (VAEs)



MNIST Image Dataset



Why Autoencoders Matter

- Compress data making large files smaller without losing important information
- Denoise by cleaning up images or data by removing unwanted noise
- Identify key patterns or features in data for tasks like clustering

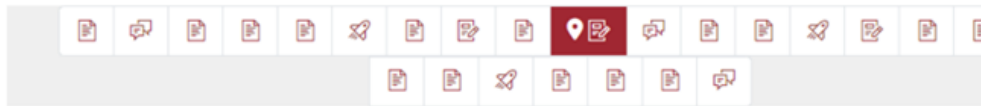
Real-Life Example

MNIST Dataset where autoencoders can shrink 256-pixel images of handwritten digits into just two features. Even with this compression, similar digits are effectively grouped together.

Variational Autoencoders (VAEs)

- Extend autoencoders by creating a "cloud" of possible data points
- VAEs generate new, realistic data that follows the same patterns as the original but isn't an exact copy

Explore the Implementation of Autoencoders in Python [Jupyter Notebook Activity 3.2]



Learning Outcome Addressed

4. Implement deep learning techniques in Python

This is a self-study activity and does not count toward the completion of the program.

Task

In this Jupyter Notebook activity, you will use the skills you've learned in the videos and mini-lesson to reconstruct the implementation of autoencoders given a scenario and dataset. This activity will prepare you to transfer the learning to more complex tasks.

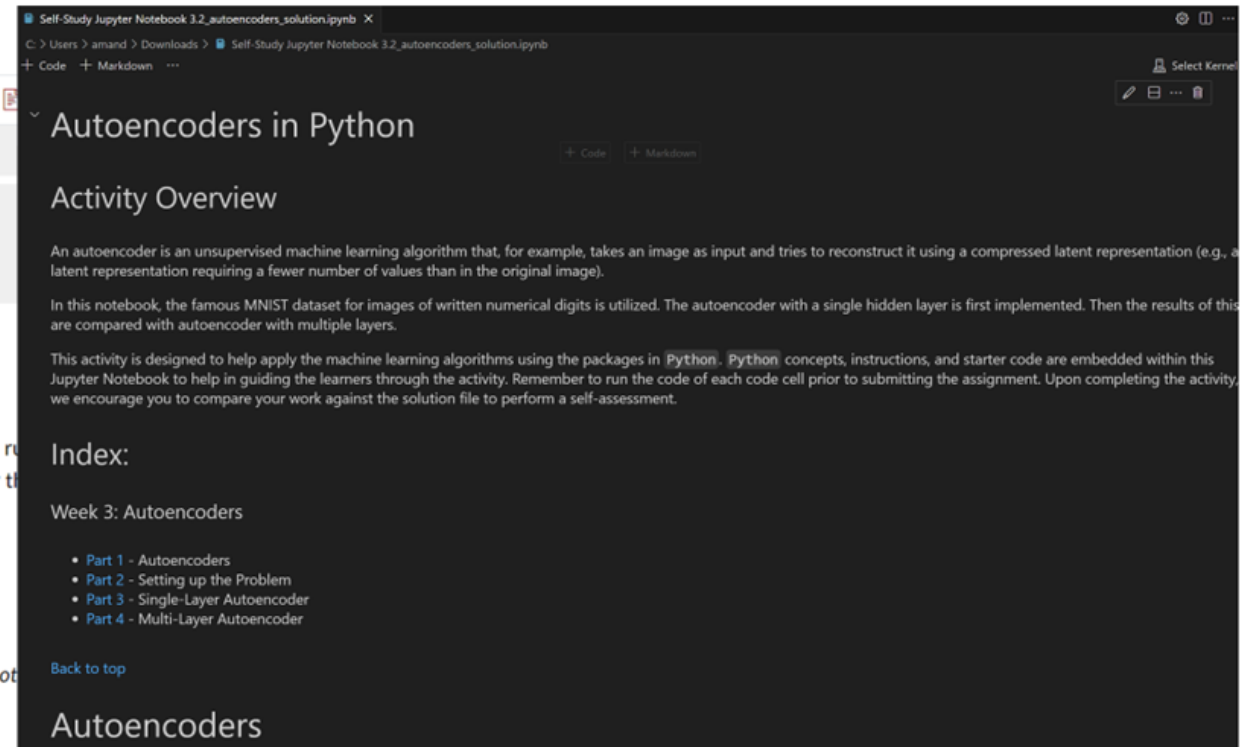
Submission Instructions

The average completion time for this activity is 30 minutes.

- After opening the activity in Codio, complete and run the code following the instructions in the Jupyter Notebook Activity.
- Important: There is nothing to submit in Codio or Canvas for this practice assignment.
- Once you've tried the activity, you can view the solution in [Google Colab](#). Find a guide to viewing your solution in Google Colab [here](#).
 - Download the solution here: [Self-Study Jupyter Notebook 3.2 Explore the Implementation of Autoencoders in Python](#)

This tool needs to be loaded in a new browser window

Load Explore the Implementation of Autoencoders in Python [Jupyter Notebook Activity 3.2] in a new window



Self-Study Jupyter Notebook 3.2_autoencoders_solution.ipynb

C:\Users\amand> Downloads > Self-Study Jupyter Notebook 3.2_autoencoders_solution.ipynb

+ Code + Markdown ...

Select Kernel

Autoencoders in Python

+ Code + Markdown

Activity Overview

An autoencoder is an unsupervised machine learning algorithm that, for example, takes an image as input and tries to reconstruct it using a compressed latent representation (e.g., a latent representation requiring a fewer number of values than in the original image).

In this notebook, the famous MNIST dataset for images of written numerical digits is utilized. The autoencoder with a single hidden layer is first implemented. Then the results of this are compared with autoencoder with multiple layers.

This activity is designed to help apply the machine learning algorithms using the packages in `Python`. `Python` concepts, instructions, and starter code are embedded within this Jupyter Notebook to help in guiding the learners through the activity. Remember to run the code of each code cell prior to submitting the assignment. Upon completing the activity, we encourage you to compare your work against the solution file to perform a self-assessment.

Index:

Week 3: Autoencoders

- Part 1 - Autoencoders
- Part 2 - Setting up the Problem
- Part 3 - Single-Layer Autoencoder
- Part 4 - Multi-Layer Autoencoder

[Back to top](#)

Autoencoders



This page contains one try-it activity and one discussion. Please scroll down to access the activities.

Self-Study Try-It Activity 3.1: Evolution of a Neural Network [15 minutes]

Learning Outcome Addressed

5. Analyze simulated neural networks.

This is a self-study activity and does not count toward the completion of the program.

Context

In this try-it activity, you will experiment with neural networks by changing the number of neurons, learning rate, and activation to observe how these changes affect the behavior of the neural network.

Introduction

[Neural Networks: Playground Exercises](#) contains a complete exercise where you can explore the different behaviors of a neural network by tuning various parameters.

Your goal is to observe the evolution of at least two different neural networks simulated by tuning various parameters.

Be sure to read the instructions at the bottom of the web page before starting the activity for optimal results.

Focus on changing the number of neurons in the hidden layer, learning rate, and activation.

Task

Follow the instructions below to test the Neural Network playground:

1. Go to the website [Neural Networks: Playground Exercises](#).
2. Begin with the first exercise: A First Neural Network.
3. Starting with Task 1, make adjustments to the neural network and view the results.
4. Consider the question in Task 1.
5. To view the answer to the Task 1 question, scroll down below the network activity to find answers to each of the questions.
6. Select the appropriate icon for an answer to Task 1.
7. Repeat this process for the remaining Tasks 2, 3, and 4.

Feel free to explore the second exercise, Neural Net Initialization, by making adjustments to the network and viewing the results. You can check your answers by scrolling to the plus icons below the activity.

After you've finished working through the first two exercises, scroll to the bottom of the page and select the icon for possible answers. This will open a video explanation that contains the possible answers and approaches to solving all the exercises.

The average completion time for this activity is 15 minutes.

<https://developers.google.com/machine-learning/crash-course/neural-networks/interactive-exercises>

Self-Study Discussion 3.1: Neural Networks: Playground [20 minutes]

Learning Outcome Addressed

5. Analyze simulated neural networks.

This is a self-study discussion and does not count toward the completion of the program.

Instructions

In this discussion, you will consider results from the preceding try-it activity and draw conclusions about how neural networks work.

In the previous activity, Self-Study Try-It Activity 3.2: Evolution of a Neural Network, you trained and simulated different neural networks. You were given the opportunity to experiment with linear and nonlinear applications within a network. You also explored and tuned various other hyperparameters.

Task

Describe the necessary difference a network needs to have in order to handle linearities and nonlinearities. What changes would you expect by changing the number of neurons in the hidden layer?

Prompts:

Read the statements posted by your peers. Feel free to respond to your peers with thoughtful comments and questions to deepen the discussion.

Submission Instructions

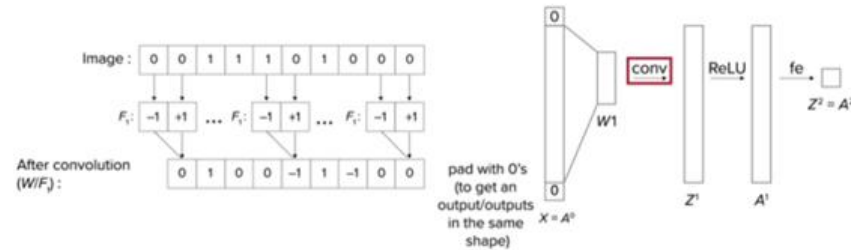
- The suggested word count for this discussion is 100-150 words.
- The average completion time for this activity is 20 minutes.



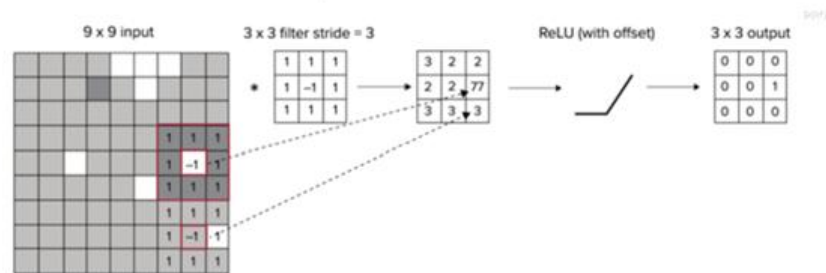
Convolutional Neural Networks (CNNs)



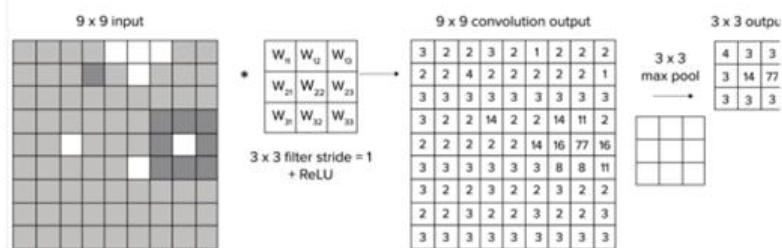
Example: 1D convolution



Example: 2D convolution



Max Pooling Layers



Convolutional Neural Networks (CNNs) Overview:

1D Convolution:

- Operates on 1D data (e.g., time series, text sequences).
- Uses a filter to perform a convolution, akin to a weighted moving average.
- Padding:** Adds zeros to input edges to maintain output size.
- ReLU Activation:** Turns negative values into zeros, adding non-linearity to model complex patterns.

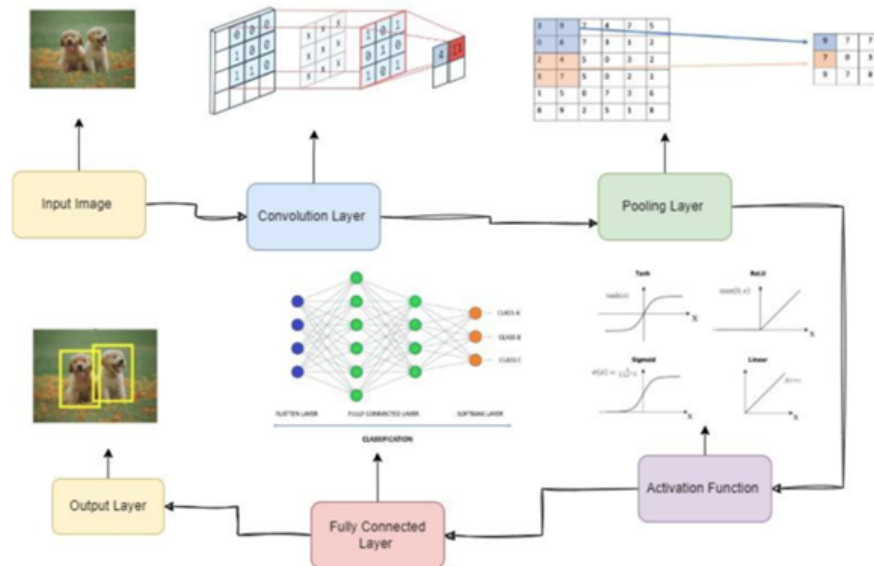
2D Convolution:

- Operates on 2D data (e.g., images).
- A 3x3 filter slides over a 9x9 image, producing a single output for each region.
- Stride:** Controls how far the filter moves with each step, affecting output size.
- ReLU Activation:** Applied after convolution with an offset for the final output.

Max Pooling Layers:

- Down-sampling operation to reduce the spatial dimensions of the output.
- Pooling Process:** The filter selects the maximum value within each region, retaining important features while reducing size.
- Helps in making the network efficient and avoiding overfitting.

Components of a Convolutional Neural Network (CNN)



Theoretical Understanding of Convolutional Neural Network: Concepts, Architectures, Applications, Future Directions

<https://www.mdpi.com/2079-3197/11/3/52>

Input Image

- Image represented as a matrix of pixel values (tiny dots of brightness and color)

Convolution Layer

- Filters slide over the image, creating feature maps by highlighting patterns like edges and textures (magnifying glasses searching for key details)

Pooling Layer

- Reduces the spatial dimensions of feature maps, keeping the most important information (zooming out while keeping key details)

Activation Function

- Introduces non-linearity, enabling the model to learn complex patterns (helps the model understand intricate details like curves)

Fully Connected Layer

- Flattens the feature maps and connects all neurons to make predictions (combines all details to determine what's in the image, e.g., recognizing a dog)

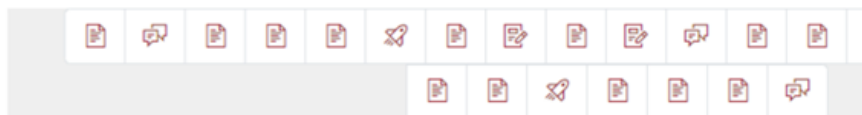
Output Layer

- Produces the final prediction or classification, adjusting the model based on errors during training (model makes a final guess and learns to improve accuracy)

Summary

- CNNs are smart systems that learn to recognize and classify images
- They work by breaking down an image into its important parts, analyzing those parts, and making a prediction about what the image shows
- Over time, CNNs get better at recognizing images, becoming very accurate in tasks like identifying objects, animals, or even faces

Explore the Implementation of CNNs in Python [Jupyter Notebook Activity 3.3]



Learning Outcome Addressed

4. Implement deep learning techniques in Python.

This is a self-study activity and does not count toward the completion of the program.

Task

In this Jupyter Notebook activity, you will put your knowledge about CNNs into practice. This Codio activity provides a scenario and dataset, and it will guide you through the steps of implementing a CNN.

Submission Instructions

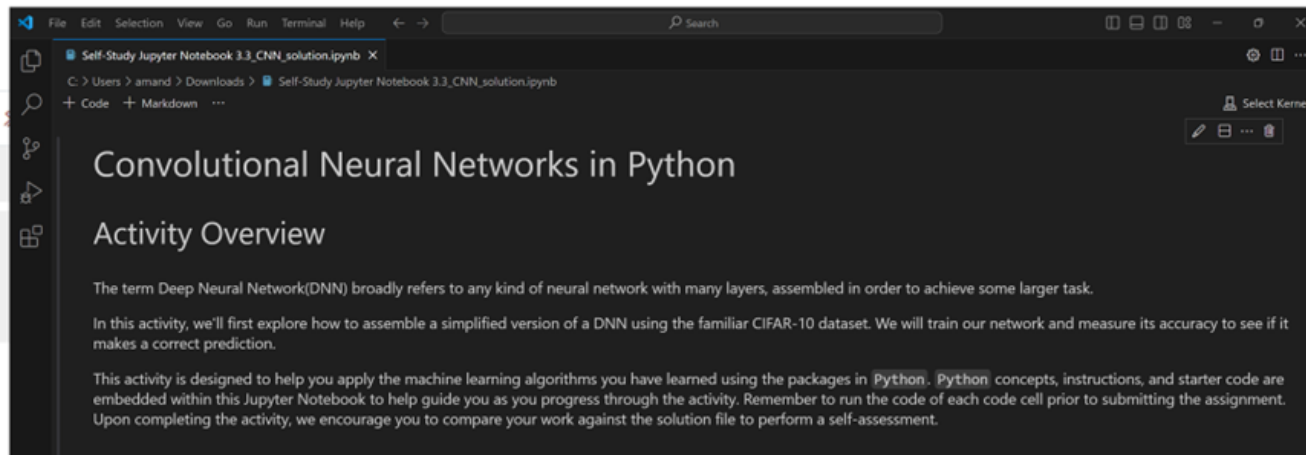


The average completion time for this activity is 30 minutes.

- After opening the activity in Codio, complete and run the code following the instructions in the Jupyter Notebook Activity.
- Important: There is nothing to submit in Codio or Canvas for this practice assignment.
- Once you've tried the activity, you can view the solution in [Google Colab](#). Find a guide to viewing your solution in Google Colab [here](#).
 - Download the solution here: [Self-Study Jupyter Notebook 3.3: Explore the Implementation of CNNs in Python](#) 
-

This tool needs to be loaded in a new browser window

Load Explore the Implementation of CNNs in Python [Jupyter Notebook Activity 3.3] in a new window



Neural Network vs. Artificial Neural Network (ANN)



Neural Network	Artificial Neural Network (ANN)
General Concept: A broad term for algorithms inspired by the human brain.	Specific Type: A type of neural network using interconnected layers of nodes.
Data Flow: Can have complex structures and data flow, including feedback loops.	Feed-Forward: Information flows in one direction, from input to output.
Learning: Can be used for a wide range of learning tasks, including deep learning.	Improvement with Input: Learns and improves with each new input, similar to how the brain learns from experience.
Applications: Used in various tasks like image recognition, predictive modeling, and natural language processing.	Specialized Use: Particularly effective for tasks with limited data and non-image inputs.
Deep Learning Foundation: Serves as the foundation for deep learning algorithms.	Deep Neural Networks: When ANNs have multiple layers, they become deep neural networks, central to deep learning.



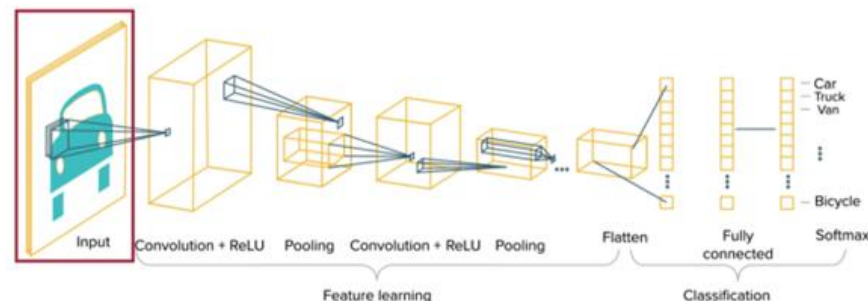
Deep Neural Networks (DNN's)

Challenges with DNNs:

- An enormous number of degrees of freedom in terms of the number of weights or model parameters
- Overfitting due to insufficient different and representative training samples

Challenges with DNNs:

- Requiring enormous numbers of training samples
- Large models are expensive to train and require many epochs or cycles of training
- A large number of hyperparameters may need to be tried in order to obtain a good or optimal result



Source: "Convolutional Neural Network 3 things you need to know" MathsWorks.
<https://in.mathworks.com/discovery/convolutional-neural-network-matlab.html>. © Property of MathWorks. Used with permission.

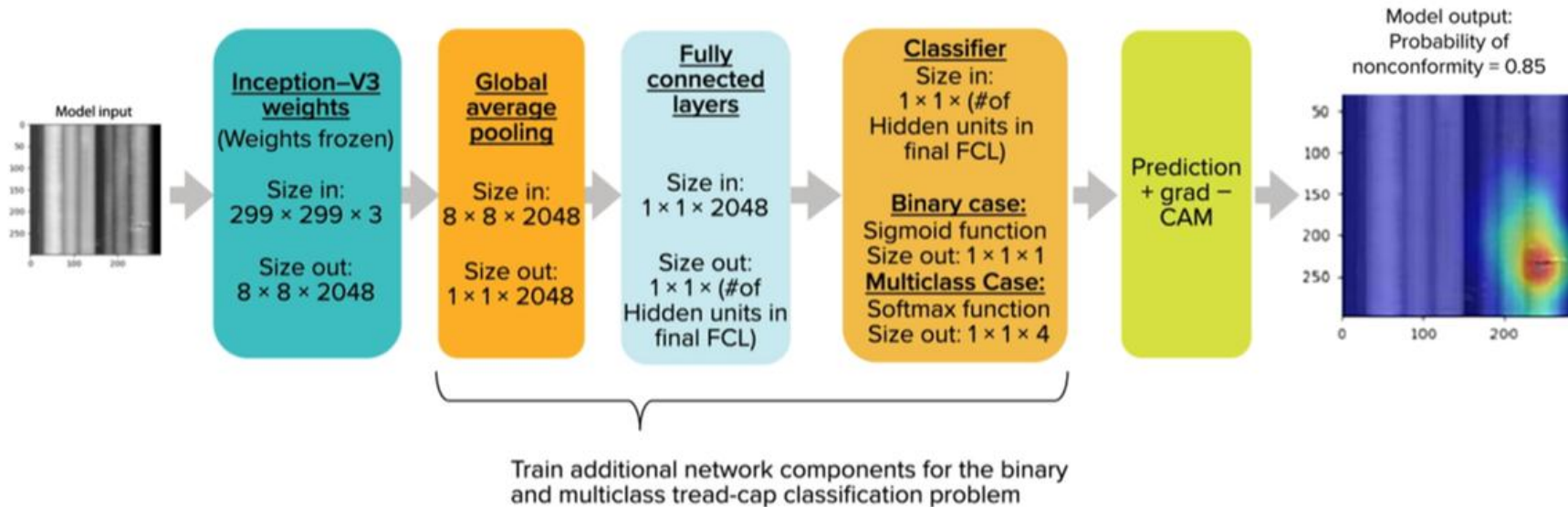
MIT



Transfer Learning

Using pretrained parameters from a different task

What are some examples of transfer learning you have seen?



Source: J.Smith, LGO Masters Thesis, MIT 2018

MIT xPRO

Self-Study Drag-and-Drop Activity 3.2: Components of Deep Neural Networks [10 minutes]

Learning Outcome Addressed

2. Identify the components of deep learning networks.

This is a self-study activity and does not count toward the completion of the program.

Task

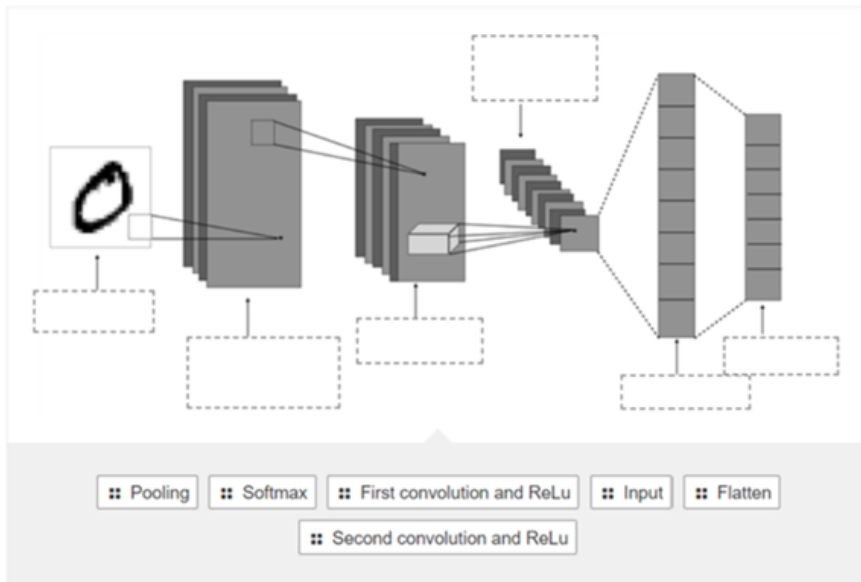
In this activity, you will identify the parts of DNNs, using the knowledge learned from Professor Boning's video.

The average completion time for this activity is 10 minutes.

Student View

[Edit question list](#)[Reports](#)

 You are viewing this activity in preview mode. Results will not be saved.



Self-Study Drag-and-Drop Activity 3.3: Characteristics and Applications of Neural Networks [10 minutes]

Learning Outcome Addressed

7. Categorize neural network algorithms.

This is a self-study activity and does not count toward the completion of the program.

Task

In this activity, you will distinguish between the different types of deep learning by using the characteristics of each to match with the correct type of neural network.

Convolutional neural networks (CNN), deep neural networks (DNN), and recurrent neural networks (RNN) can be distinguished from each other based on their structure, characteristics, and applications. This activity is designed for you to consolidate your knowledge about the differences.

The average completion time for this activity is 10 minutes.

Student View

[Edit question list](#)[Reports](#)

 You are viewing this activity in preview mode. Results will not be saved.



Discriminate between Different Types of Neural Networks

1 of 2



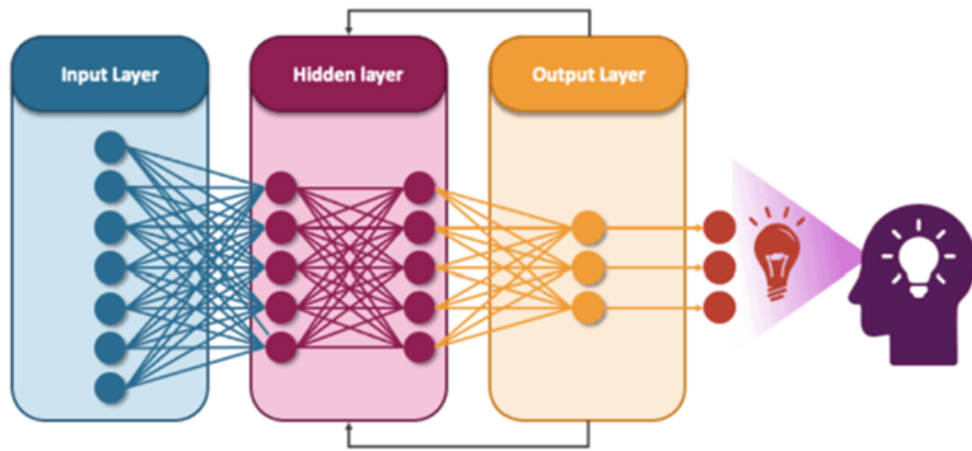
CNNs	DNNs	RNNs
⌘ The result of a layer for this NN can be passed to an MLP. ⌘ They are used after a convolution. ⌘ The output is fed back into the network to enable modelling. ⌘ They are NNs with multiple layers. ⌘ They deal with sequence data. ⌘ They work in combination with multiple steps of convolution, ReLu, and pooling. ⌘ A key component of this algorithm is backpropagation.		

Check Answer

Question 1

Question 2

Recurrent Neural Networks (RNN's)



Source: <http://vinodsblog.com>

Recurrent Neural Networks (RNNs)

- Designed to handle sequence data with temporal dependencies
- Maintains memory of previous inputs for context understanding
- **Applications**
 - Speech recognition, language translation, text generation
- **Challenges**
 - Struggles with long-term dependencies; addressed by LSTM and GRU variants.

Neural Network Structure

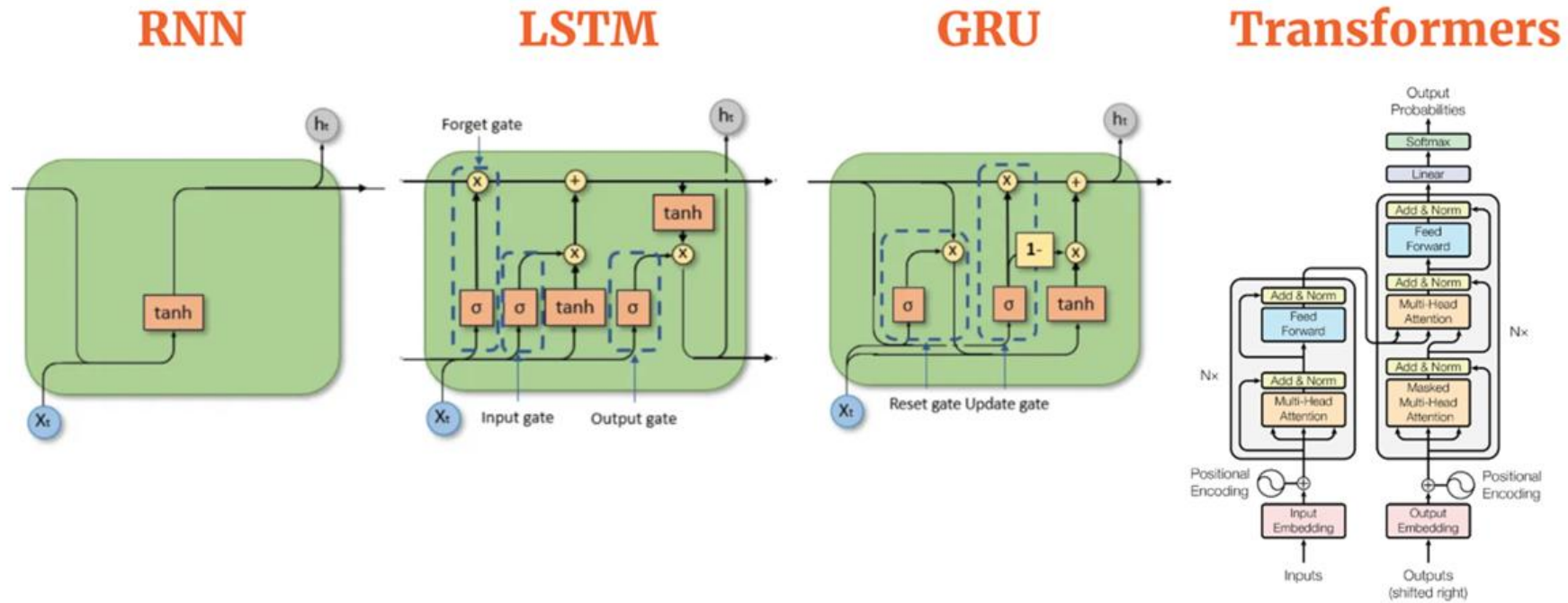
- **Input Layer**
 - Receives input data (e.g., image pixels)
- **Hidden Layers**
 - Learns patterns and adjusts weights
- **Output Layer**
 - Generates final prediction or output
- **Connections**
 - Weighted links between layers; data flows from input to output
- **Feedback Loop**
 - Iteratively refines predictions to improve accuracy



	ANN	CNN	RNN
Basics	One of the simplest types of neural networks.	One of the most popular types of neural networks.	The most advanced and complex neural network.
Structural Layout	Its simplicity comes from its feed forward nature – information flows in one direction only.	Its structure is based on multiple layers of nodes including one or more convolutional layers.	Information flows in different directions, which gives it its memory and self-learning features.
Data Type	Fed on tabular and text data.	Relies on image data.	Trained with sequence data.
Complexity	Simple in contrast with the other two models.	Considered more powerful than the other two.	Fewer features than CNN but powerful due to its self-learning & memory potential.
Commendable Feature	Ability to work with incomplete knowledge and high fault tolerance.	Accuracy in recognizing images.	Memory and self-learning.
Feature type: spatial recognition	No	Yes	No
Feature type: Recurrent connections	No	No	Yes
Main Drawback	Hardware dependence.	Large training data required.	Slow and complex training and gradient concerns.
Uses	Complex problem solving such as predictive analysis.	Computer vision including image recognition	Natural language processing including sentiment analysis and speed recognition.

<https://levity.ai/blog/neural-networks-cnn-ann-rnn>

Comparing ANN's, CNN's, and RNN's



Title: Comparing different Sequence models: RNN, LSTM, GRU, and Transformers

Source: Colah's blog, and Attention paper. Compiled by AIML.com Research

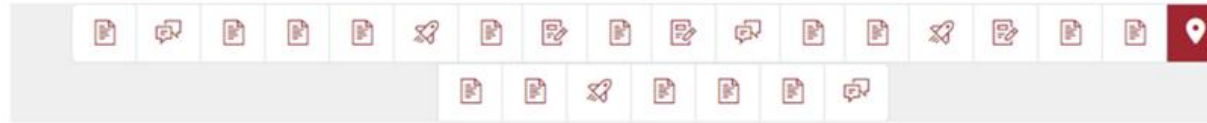
Comparing RNN's, LSTM's, GRU's, and Transformers



Description	Recurrent Neural Network (RNN)	Long Short Term Memory (LSTM)	Gated Recurrent Unit (GRU)	Transformers
Overview	RNNs are foundational sequence models that process sequences iteratively, using the output from the previous step as an input to the current step.	LSTMs are an enhancement over standard RNNs, designed to better capture long-term dependencies in sequences.	GRUs are a variation of LSTMs with a simplified gating mechanism.	Transformers move away from recurrence and focus on self-attention mechanisms to process data in parallel
Key characteristics	- Recurrent connections allow for the retention of "memory" from previous time steps.	- Uses gates (input, forget, and output) to regulate the flow of information. - Has a cell state in addition to the hidden state to carry information across long sequences. © AIML.com Research	- Contains two gates: reset gate and update gate. - Merges the cell state and hidden state.	- Uses Self-attention mechanisms to weigh the importance of different parts of the input data. - Consists of multiple encoder and decoder blocks. - Processes data in parallel rather than sequentially.
Advantages	- Simple structure. - Suitable for tasks with short sequences.	- Can capture and remember long-term dependencies in data. - Mitigates the vanishing gradient problem of RNNs.	- Fewer parameters than LSTM, often leading to faster training times. - Simplified structure while retaining the ability to capture long-term dependencies.	- Can capture long-range dependencies without relying on recurrence - Highly parallelizable, leading to faster training on suitable hardware.
Disadvantages	- Suffers from the vanishing and exploding gradient problem, making it hard to learn long-term dependencies - Limited memory span	- More computationally intensive than RNNs - Complexity can lead to longer training times.	- Might not capture long-term dependencies as effectively as LSTM in some tasks.	- Requires a large amount of data and computing power for training. - Can be memory-intensive due to the attention mechanism, especially for long sequences.
Use Cases	Due to its limitations, plain RNNs are less common in modern applications. Used in simple language modeling, time series prediction	Machine translation, speech recognition, sentiment analysis, and other tasks that require understanding of longer context.	Text generation, sentiment analysis, and other sequence tasks where model efficiency is a priority.	- State-of-the-art performance in various NLP tasks, including machine translation, text summarization. - Forms the backbone for models like BERT and GPT.
Model variants	Vanilla RNN, Bidirectional RNN, Deep (Stacked) RNN	Vanilla LSTM, Bidirectional LSTM, Peephole LSTM, Deep (Stacked) LSTM	GRU	Original Transformer (Seq-to-Seq), Encoder only (Eg: BERT), Decoder only (Eg: GPT), Text to Text (Eg: T5)

<https://aiml.com/compare-the-different-sequence-models-rnn-lstm-gru-and-transformers/>

Explore the Implementation of RNNs in Python [Jupyter Notebook Activity 3.4]



Learning Outcome Addressed

4. Implement deep learning techniques in Python.

This is a self-study activity and does not count toward the completion of the program.

Task

In this Jupyter Notebook activity, you will put your knowledge about RNNs into practice. You will be given a scenario and a dataset, and you will implement RNNs using this step-by-step guide.

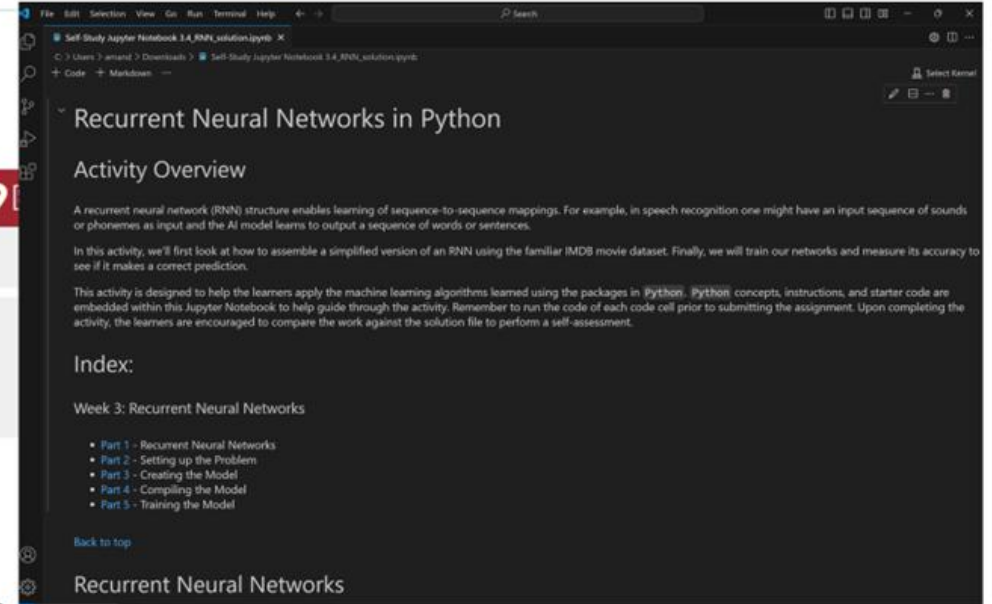
Submission Instructions

The average completion time for this activity is 30 minutes.

- After opening the activity in Codio, complete and run the code following the instructions in the Jupyter Notebook Activity.
- Important: There is nothing to submit in Codio or Canvas for this practice assignment.
- Once you've tried the activity, you can view the solution in [Google Colab](#). Find a guide to viewing your solution in Google Colab [here](#).
 - Download the solution here: [Self-Study Jupyter Notebook 3.4: Explore the Implementation of RNNs in Python](#)

This tool needs to be loaded in a new browser window

Load Explore the Implementation of RNNs in Python [Jupyter Notebook Activity 3.4] in a new window



Calculating Performance Metrics for Classification Models

Steps to calculate the accuracy, F1 score, precision, and recall based on the Confusion matrix:

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Steps:

- Accuracy
 - Add True Positives (TP) and True Negatives (TN).
 - Divide by the total number of predictions (TP + TN + FP + FN).
- Precision
 - Divide True Positives (TP) by the sum of True Positives (TP) and False Positives (FP).
- Recall
 - Divide True Positives (TP) by the sum of True Positives (TP) and False Negatives (FN).
- F1 score
 - Calculate the harmonic mean of Precision and Recall:
 - Multiply Precision and Recall.
 - Multiply the result by 2.
 - Divide by the sum of Precision and Recall.



How are you using ANN's, CNN's, RNN's, LSTM's, GRU's, Transformers, or Hybrid Models?



Neural Network Type	ANN (Artificial Neural Network)	CNN (Convolutional Neural Network)	RNN (Recurrent Neural Network)	LSTM (Long Short-Term Memory)	GRU (Gated Recurrent Unit)	Transformer	Hybrid Models (ex., CNN + RNN)
When to Use	General purpose, complex input-output relationships, no temporal or spatial data	When working with spatial data where hierarchies are important, like images or videos	Input data has sequential dependencies (ex., time series) but doesn't require long-term memory retention	Long sequences where long-term memory is crucial, solving the vanishing gradient problem	Like LSTM but more efficient for shorter sequences or faster training needs.	For long sequences requiring parallel processing and attention mechanisms, ideal for large datasets	When both spatial and temporal data need analysis (ex., videos)
Use Cases	- Financial forecasting - Medical diagnosis - Fraud detection - Customer churn prediction - Quality control in manufacturing	- Image classification (ex., facial recognition) - Image segmentation (ex., medical imaging) - Video analysis - Handwriting recognition - Document classification	- Text generation (ex., auto-completion) - Time series forecasting - Basic language modeling - Sentiment analysis (short text) - Sequential recommendation systems	- Text generation (ex., story writing) - Speech recognition - Machine translation - Stock market prediction - Music generation - Video captioning	- Speech recognition - Real-time sensor data prediction - Time series forecasting - Predictive text input - Video streaming recommendations	- NLP (ex., language translation, question-answering) - Text summarization - Named Entity Recognition (NER) - Document classification - Chatbots - Protein structure prediction	- Video classification - Image captioning in videos - Satellite weather forecasting - Autonomous driving (ex., path planning, obstacle detection)





Questions?